



Fev 2026



Processadora de cartões: atingindo o 60/30/30



Cássio Gröger
Executivo de Contas



Elúzio Barretto
Arquiteto de Soluções



Leonardo Campiolo
Diretor Regional



Brenda Guimarães
Desenvolvimento de novos negócios

partnership to execute the

60 30 30



Total Clients ~60MM

“Valuable connection with clients translates into **brand power**”

Efficiency Ratio ~30%

“Our continued focus on identifying opportunities for **efficiency improvements**”

Return on Equity ~30%

“Growing credit with focus on **ROE and long-term sustainability**”



Noted for its flexibility, compliance regulation, and **high level of scalability** as an application data platform, MongoDB Atlas helped PicPay expand and maintain its competitive edge.



With majority of new developments are built on MongoDB Atlas Agibank significantly reduced time to market, costs **by 90%**, and improved performance **by 4.8x faster**.



By shifting from manual processes to an **AI-driven** solution, Base39 cut loan analysis costs by **96%** and reduced infrastructure expenses by **84%**.





200M embedded finance loans and **100M** retail accounts at a record-breaking **150k** transactions per second. In doing so, Temenos proved its robust and scalable platform can support banks' business models



By migrating from Amazon DocumentDB & Elasticsearch to MongoDB Atlas resulting on new application features **releases 90% faster**.



The personalized products Wells Fargo can offer are built into the data **they have on every transaction** and the knowledge they have on their customer

Resumo do Projeto de Cartões



Objetivos Corporativos:

60M de clientes
30% de índice de eficiência
30% de ROE

Estratégias de Negócios:

Crescimento em cartão crédito, expandindo ROE e diversificando receitas

Iniciativas de TI:

1. Inovação: crédito (que inclui cartões) tem impulsionado o ROE.
2. Excelência Operacional/Execução impecável e FinOps: Buscando o índice de eficiência, saindo do Oracle para postgre que tem altos custos.

Desafios:

1. **Gargalos de processamento:** Histórico de dificuldades com inserções massivas (1 hora de processamento noturno), o cliente as vezes acessa o fechamento de fatura nesse momento e gera reclamações (BACEN e time de atendimento ao cliente)
2. **Volumetria Massiva e criticidade:** Gerenciamento de milhões de dados permanentes (clientes, faturas, PDFs) que exigem alta disponibilidade, e rapidez inconsistências momentâneas geram carga no time de atendimento ao cliente e reclamações formais ao BACEN
 - a. Para atender o cliente rapidamente cria um cache que gera complexidade arquitetural e episódios de inconsistência
3. **Gestão Financeira:** O custo é uma pauta diária e "100% envolvida" na arquitetura das aplicações, com comitês de acompanhamento a cada 15 dias, e aumentar a performance muitas vezes tem a limitação tecnológica e financeira.
4. **Escalabilidade:** Necessidade de adicionar nós conforme a demanda cresce, o que impacta diretamente o custo linear da operação.
5. Conciliação: precisa conseguir trabalhar no processo de conciliação, vai ser natural a evolução dos modelos de dados nesse sentido
6. Experiência do cliente: permitir ao cliente fazer busca de transações (categorizar)

Capacidades Necessárias para abordar esses desafios:

1. Maior Performance em com custo alinhado ao benefício
 - a. atendendo picos de demandas de backoffice ou do cliente.
2. Simplificar componentes (fim do caching)
3. Rápido desenvolvimento da primeira versão e evolução buscando a conciliação e busca textual
4. Prontidão para Real-time das transações

Como fazemos isso / Como fazemos isso melhor:

1. Banco de dado otimizado para cartões e para operações do tipo: se vc lê junto escreve junto.
 - a. Escalabilidade nativa programada ou automática sem downtime.
2. Atender cliente em cerca de 10-20 ms é capaz de não necessitar do cache, eliminando ETLs e ciclos maiores de desenvolvimento em releases
3. JSON nativo (Informação já chega em JSON do SQS), sem downtime/alerttable e desafios ao evoluir o esquema, com text search nativo.

Situação futura e Resultado de negócios esperados:

Redução de custos em X%, representando \$
Redução de tempo de processamento em Y%
Redução dos chamados em Z%, reduzindo o CTS total em \$

Redução de dependência do COBOL e legados do Inter: Case de referência?



WELLS
FARGO

Quando a empresa decidiu lançar sua iniciativa de Cartões 2.0 para tornar a plataforma de pagamento mais sustentável e escalável, ela precisava, primeiro, **modernizar seu mainframe**. Embora os mainframes tenham dados críticos do Sistema de Registro (SOR), muitas vezes, eles trazem dívidas técnicas, dependências e são cada vez mais caros de gerenciar, além de uma escassez de pessoas com as habilidades certas para mantê-las.

Atualmente quando os **dados de um fornecedor externo entram no mainframe**, eles são **importados para o MongoDB e disponibilizados para outros canais**. A equipe criou uma camada de fachada entre seu banco de dados e APIs para habilitar microserviços sem interface gráfica que oferecem suporte para diferentes linhas de produtos dentro da empresa e fornecem mais flexibilidade do que sua arquitetura monolítica anterior.

Sugestão da MongoDB:

No Inter: Existe uma integração entre o mundo Java e sistemas legados em COBOL

O Argumento: O MongoDB pode atuar como uma "camada de aceleração". Em vez de forçar o Postgres a tentar lidar com a estrutura rígida que vem do mainframe/COBOL, o MongoDB armazena o estado atual do cartão como um documento rico (JSON->BSON), facilitando a vida dos desenvolvedores Java.



What is Cards

Supports Credit Card Platform

Cards 2.0: Support sustainable and scalable cards platform

- Ability for a CC Customer to come through any channel (branch/mobile/digital/contact center etc.) and have a seamless experience
- Better integration and unit testing for resiliency
- Reusable and Scalable APIs
- Less dependence on 3rd party processor for things that don't change, eg. statemented transactions and thereby reduce MIPS/cost
- Led to creation of ODS

Why MongoDB

- Scale - 7+ million transactions/hour
- Better Resiliency and Control
- Tighter deadlines

Escalabilidade num cenário complexo e de grande volume



WELLS
FARGO

"O MongoDB lida com mais de 20 terabytes de dados por dia e oferece suporte para mais de 80 microsserviços. No momento estamos usando o ODS para executar nossos programas de recompensas e fidelidade, por exemplo, que eram executados no SQL Server anteriormente", recorda Vemana.

"Criamos um Armazenamento de Dados Operacional (ODS) no MongoDB para resolver todos esses problemas", explica Ram Vemana, Líder de Dados de Cartão de Crédito e Comércio do Wells Fargo, durante sua sessão de debate com o MongoDB em Nova York, em junho. "Ele tem um serviço de menos de um segundo que pode lidar com mais de sete milhões de transações. Isso é mais do que o suficiente para hoje e fornece a escalabilidade e resiliência das quais precisamos para o futuro.

Sugestão da MongoDB:

No Inter: O time citou "gargalos de inserção massiva" e a necessidade de nós separados nos bancos.

O Argumento: Temos cases em que o MongoDB aguenta o tráfego em um cenário complexo e de alto volume. Se o Inter planeja continuar crescendo a base de cartões, a escalabilidade nativa do MongoDB pode mitigar gargalos da operação.



Banks issue the cards (Visa/Mastercard), but the gateway that tracks all the transactions is the Fiserv mainframe system

Card processors send the Nightly files to banks which are typical Legacy Mainframe Applications

Operational data store, built on MongoDB to address high MIPS cost of Mainframes

Files are sent to FI with Card statements, billing cycle info, etc nightly.

Operational Data Store(ODS) is build using the nightly files coming from SOR

This led to easy access for SOR data on more modern techstack for Product lines to be able to Launch new products and Apps with out dependency on processor

ODS can also be made in sync with processor using the realtime feed from SOR for critical updates happens in business hours.

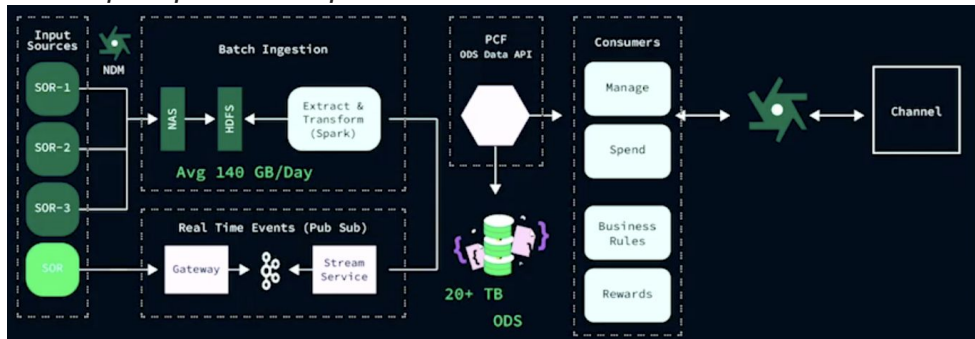
Operational Data Store

Hub de API do Inter: Case de referência?

**WELLS
FARGO**

A visão dos Cartões 2.0 era uma experiência multicanal tranquila para os consumidores. O Wells Fargo queria reduzir a dependência do processamento de terceiros tanto quanto possível, simplificar as integrações e adotar APIs reutilizáveis para que pudesse implementar rapidamente as alterações em todos os canais.

"Criamos um Armazenamento de Dados Operacional (ODS) no MongoDB para resolver todos esses problemas", explica Ram Vemana, Líder de Dados de Cartão de Crédito e Comércio do Wells Fargo, durante sua sessão de debate com o MongoDB em Nova York, em junho. "Ele tem um serviço de menos de um segundo que pode lidar com mais de sete milhões de transações. Isso é mais do que o suficiente para hoje e fornece a escalabilidade e resiliência das quais precisamos para o futuro."



Sugestão da MongoDB:

No Inter: Você mencionaram que a área de Processamento de Cartões atua como um hub para outros produtos.

O Argumento: Temos a hipótese Inter precisa de uma camada de dados que não "engasgue" quando múltiplas APIs (diversos produtos) lançar transções, consultar faturas e limites simultaneamente.

O MongoDB permite que esse hub seja performático sem a complexidade de JOINS que Banco Relacionais exige.

Agilidade em desenvolvimento: Case de referência?

**WELLS
FARGO**

Com o MongoDB, o Wells Fargo iniciou sua modernização de mainframe legado e está bem posicionado para continuar inovando e oferecendo aos consumidores serviços financeiros de última geração.

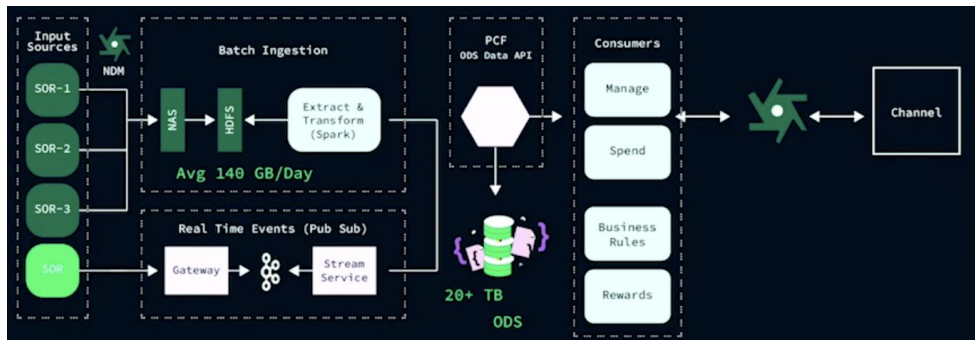
“O que estamos fazendo é ao mesmo tempo angustiante e emocionante. Os dados são uma mina de ouro. Os produtos personalizados que podemos oferecer estão integrados aos dados que temos em cada transação e ao conhecimento que temos sobre nossos clientes”, diz Kayani. “É tudo uma questão de como podemos utilizar isso para melhor atender nossos clientes.”

Sugestão da MongoDB:

No Inter: O Super App está em constante evolução.

O Argumento: Se o time de negócios do Inter decide mudar a forma como processa o cartão no MongoDB isso é uma mudança muito mais tranquila comparada ao mundo de banco relacionais que gera risco e overhead.

Além disso maioria das API usam JSON na comunicação, nativamente compatível com MongoDB



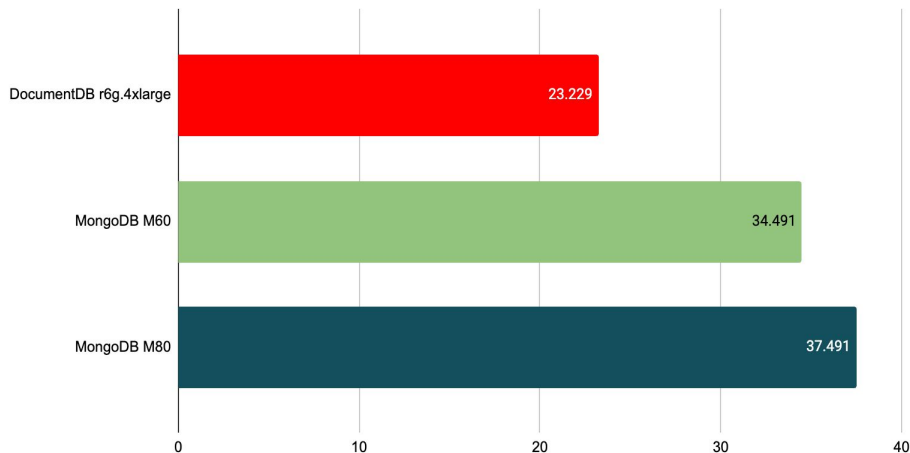
Anexos opcionais (AWS DocumentDB)

Resultados

Carga de Documentos por Segundo

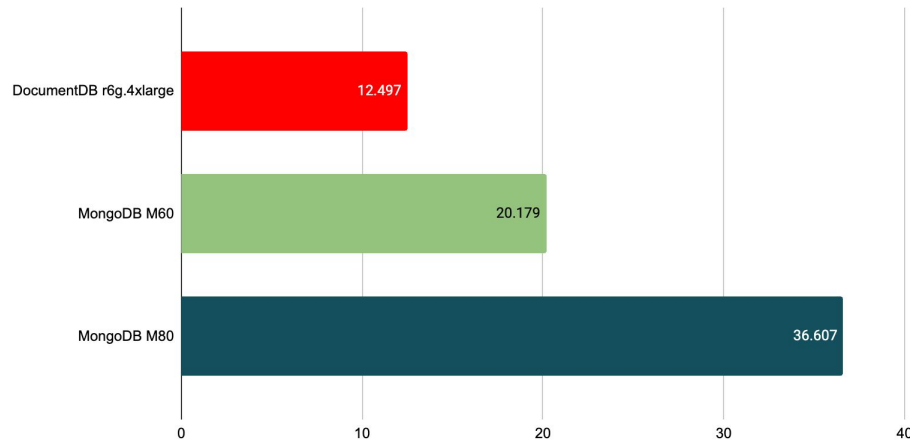


Carga de Documentos por Segundo (maior melhor)



Teste 1 (sem índices)

Carga de Documentos por Segundo (maior melhor)



Teste 2 (com índices)

Atlas M60: **1,5x** melhor que DocumentDB
Atlas M80: **1,6x** melhor que DocumentDB

Atlas M60: **1,6x** melhor que DocumentDB
Atlas M80: **3x** melhor que DocumentDB

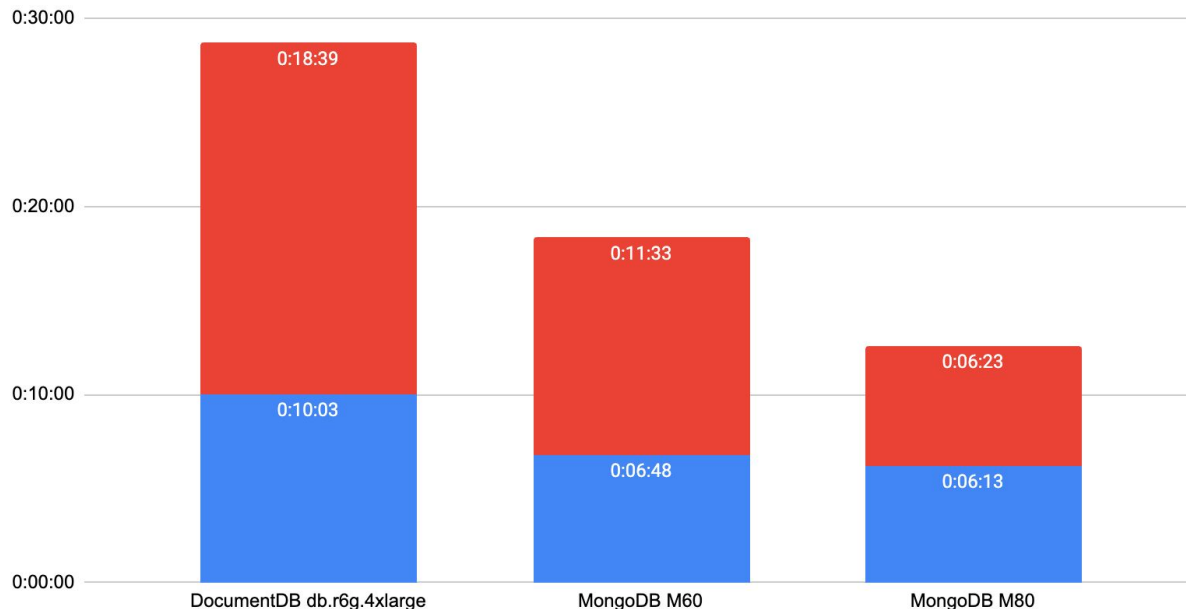


Resultados

Tempo de Carga Total

Tempo de Carga Total (menor melhor)

■ Tempo Teste 2 (com índices) ■ Tempo Teste 1 (sem índices)



Atlas M60: **1,6x** mais rápido

Atlas M80: **2,3x** mais rápido

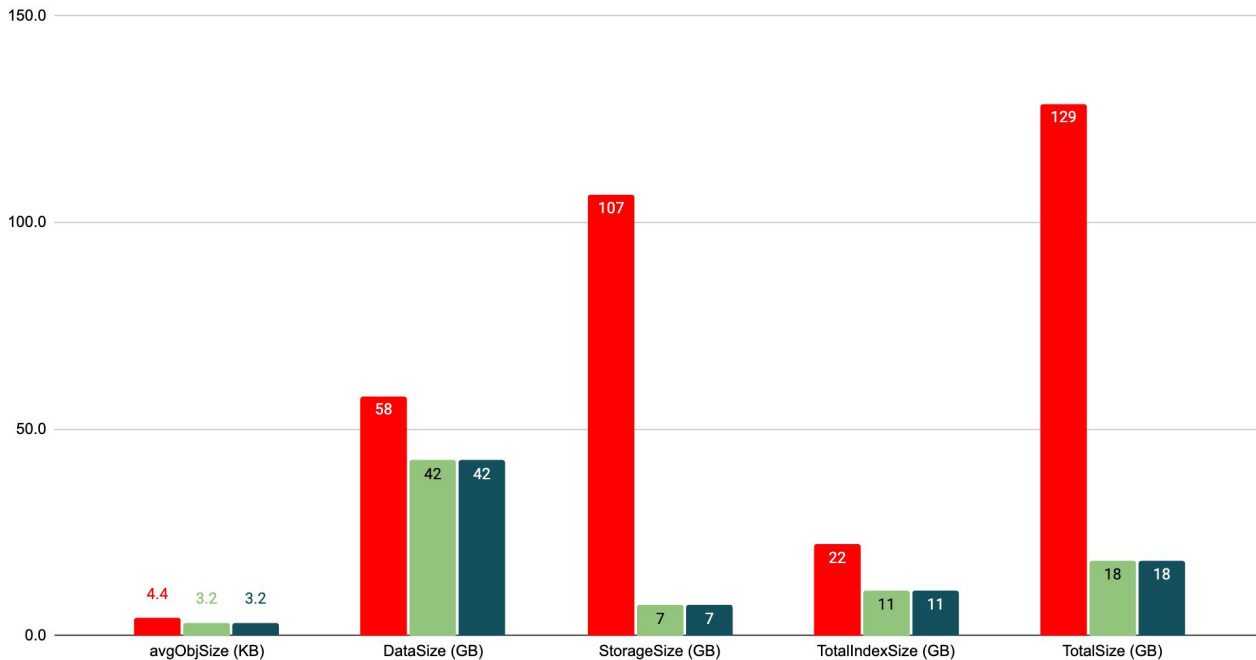
Resultados

Eficiência de Armazenamento



Armazenamento (menor melhor)

DocumentDB r6g.4xlarge MongoDB M60 MongoDB M80



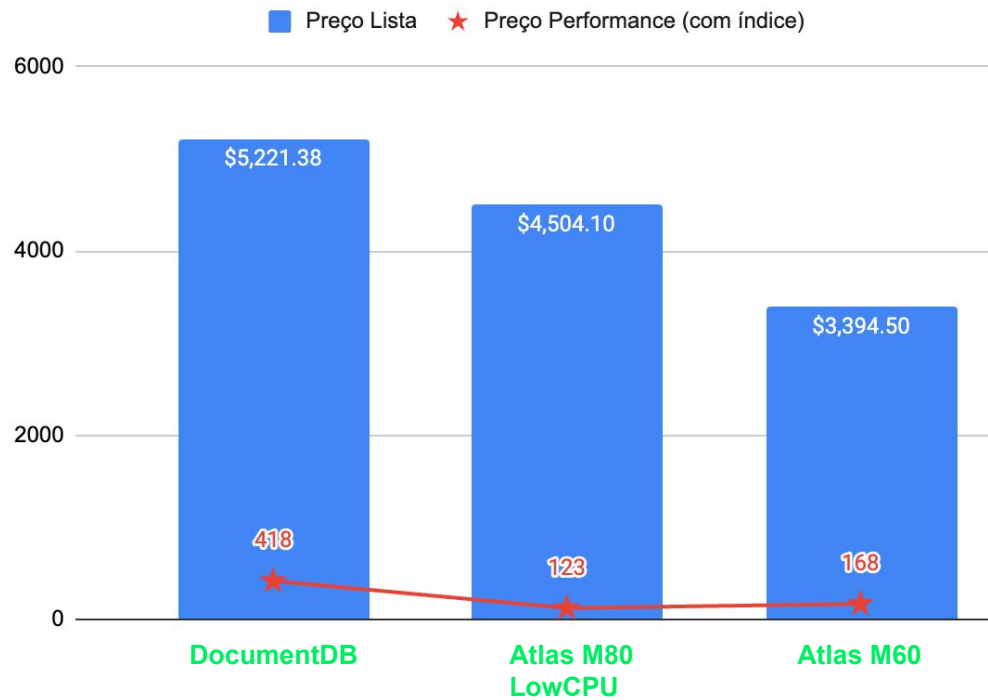
Vantagens do MongoDB:

- Documento **27%** menor
- Dados **27%** menores
- Índices **51%** menores
- Compressão de **5,7x**
- **7x** menos disco total

Comparativo de Custos



Custos	DocumentDB	Atlas M80 LowCPU	Atlas M60
Tempo/Mes	730 horas	730 horas	730 horas
Instancias	3 instancias I/O Optimized	3 instancias	3 instancias
Shape	db.r5g.4xlarge	M80 LowCpu	M60
Memória	128	128	64
vCPU	16	16	16
Região	AWS - Virginia	AWS - Virginia	AWS - Virginia
Disco	500 GB	750 GB	500 GB
Preço Lista	\$5,221.38	\$4,504.10	\$3,394.50
Preço Performance	418	123	168



Sumário dos Testes M60



	DocumentDB	MongoDB	MongoDB	
	db.r5g.4xlarge	M80	M60	Diferença M60
Tempo Teste 1 (sem índices)	0:10:02.59 (602 s)	0:06:13.45 (373 s)	0:06:48 (408 s)	0:03:14 (32% mais rápido)
Documents/second	23.229	37.491	34.491	1,5x maior
Tempo Teste 2 (com índices)	0:18:39.47 (1119 s)	0:06:22.94 (382 s)	0:11:33 (693 s)	0:07:06 (38% mais rápido)
Documents/second	12.497	36.607	20.179	1,6x maior
Document Count	13.984.000	13.984.000	13.984.000	igual
avgObjSize (bytes)	4.456	3.256	3.256	27% menor
DataSize (bytes)	62.312.704.000	45.531.904.000	45.531.904.000	27% menor
StorageSize (bytes)	114.589.442.048	7.947.939.840	7.947.939.840	93% menor (5,7x compress)
totalIndexSize (bytes)	23.727.431.680	11.571.085.312	11.571.085.312	51% menor
TotalSize	138.316.873.728	19.519.025.152	19.519.025.152	7x menor

Redução de Risco: Ganhos de continuidade de negócios



- Falhas em caso de picos de demanda: sua solução não tem escalabilidade nativa, o nosso possui



- O Fail-over não planejado de sua solução pode chegar a minuto e pode engasgar a aplicação inteira (o nosso é de segundos)

- Se você precisar fazer uma configuração de alteração (escalar o ambiente, diminuir ambiente) no DocumentDB você recebe essa mensagem:



Potential unexpected downtime

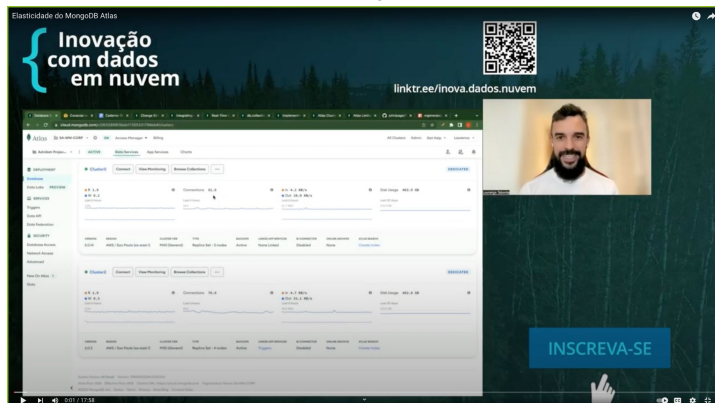
If you choose to apply changes immediately, please note that any changes in the pending modifications queue are also applied. If any of the pending modifications require downtime, choosing this option can cause unexpected downtime.

- SLA de Disponibilidade de sua solução é 99.99% e a do MongoDB Atlas (nossa solução) é de 99.995%: 41 minutos de indisponibilidade mensal de diferença entre os SLAs.

Escalabilidade sem downtime com Atlas

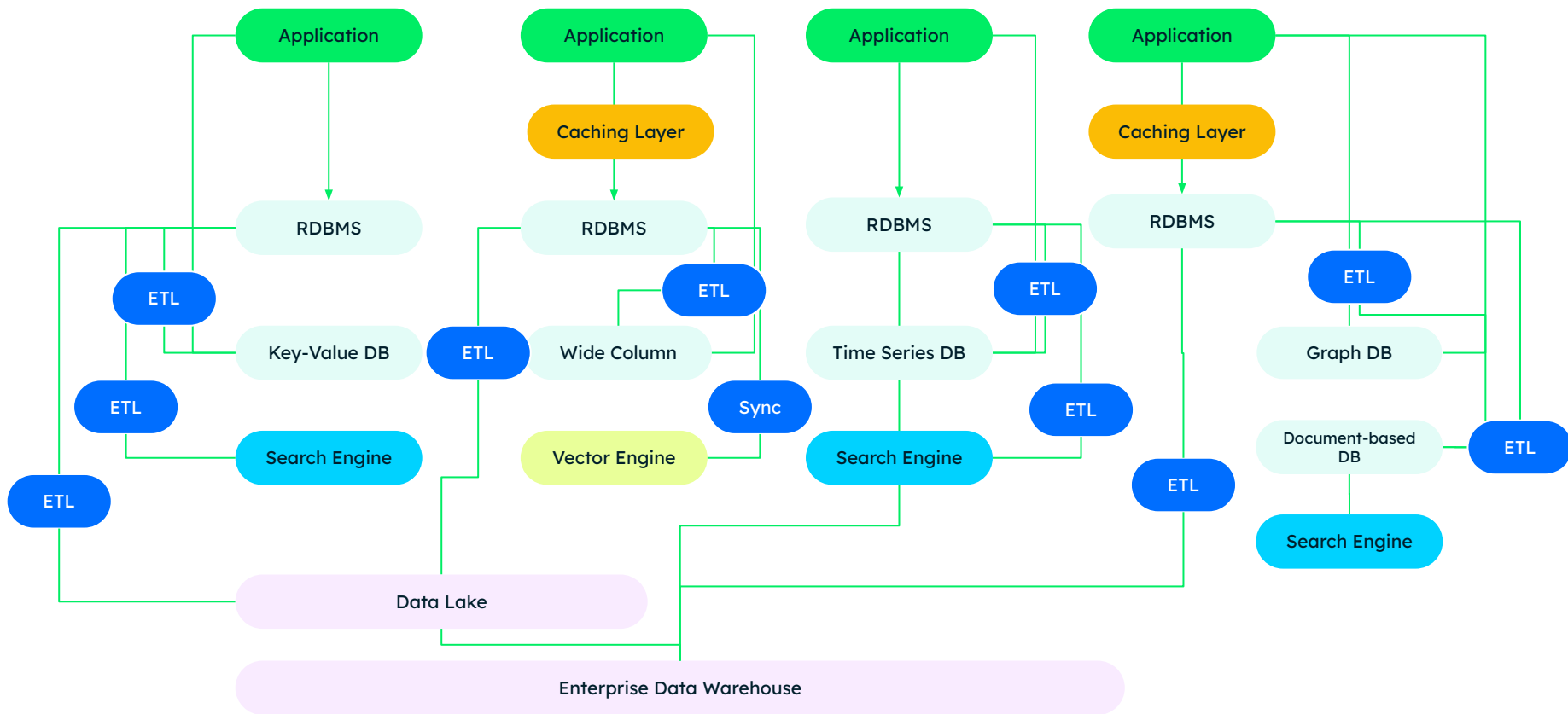


Temos um material que demonstra isso no [Youtube](#).

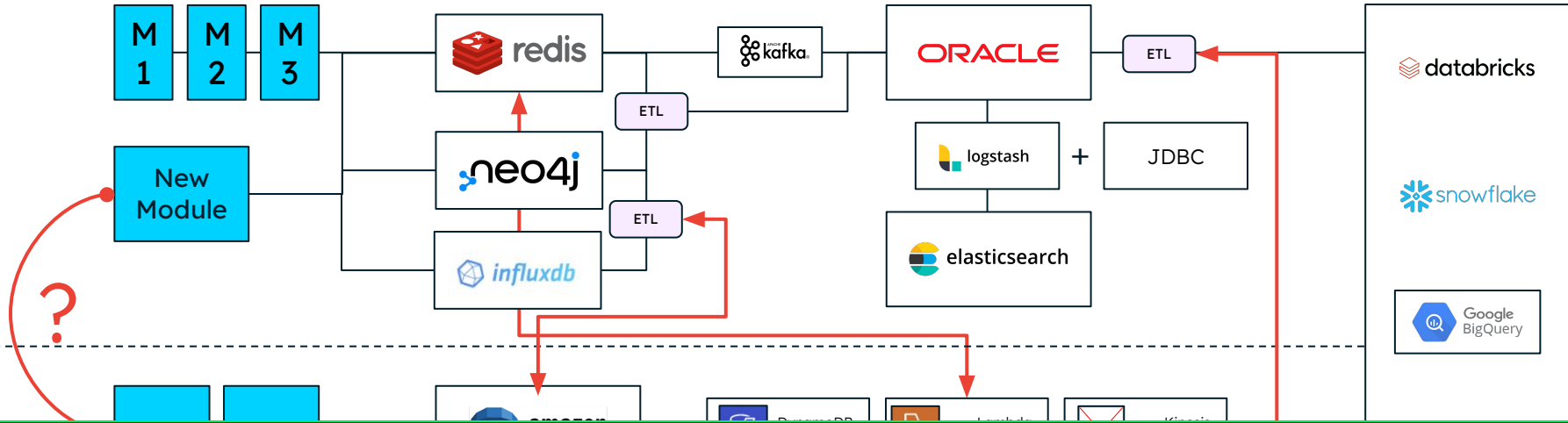




Aumento do ROE:
maior produtividade para
ganho de diferencial
competitivo na simplificação
dos componentes



How do we get the apps talking without going crazy



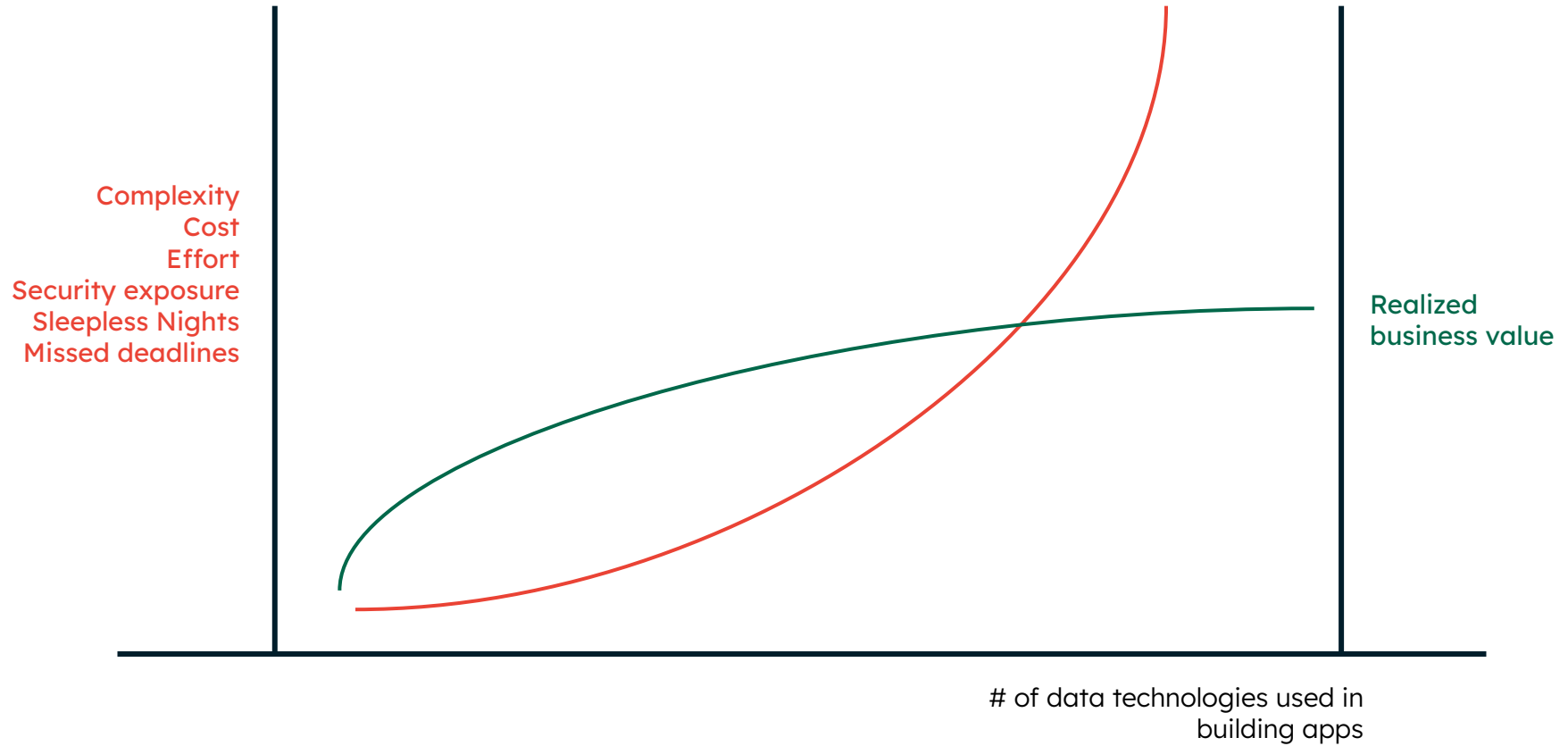
This complexity increases cost while reducing your pace of innovation

Fragmented developer experience

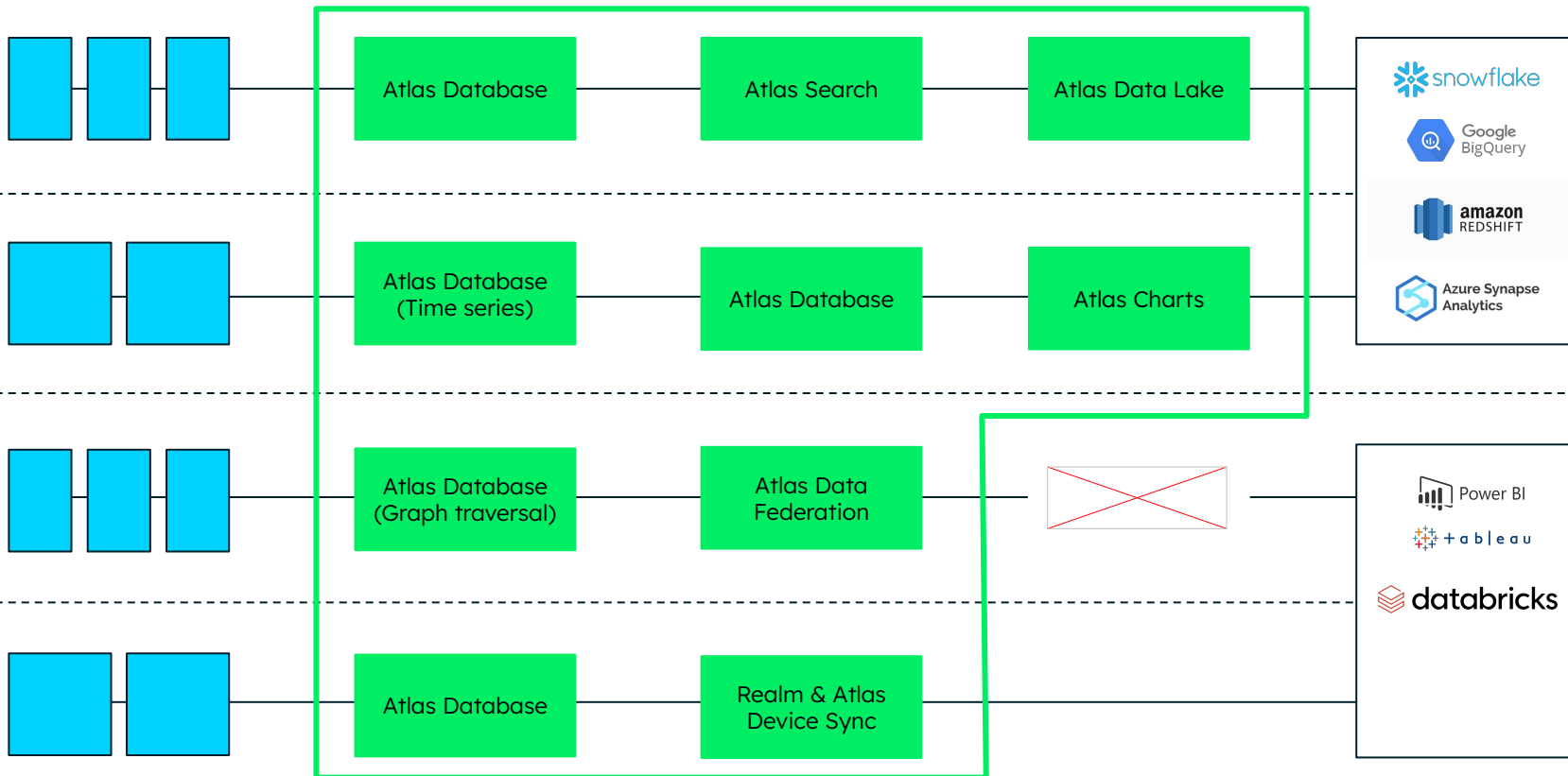
Exponential increase in number of system component interactions to plan, build, monitor, and maintain

Increased time & effort on undifferentiated work such as data movement and dealing with data duplication

No repeatable models of success



Developer Data Platform





The pain of bolting on a search engine

✗ Lower developer productivity

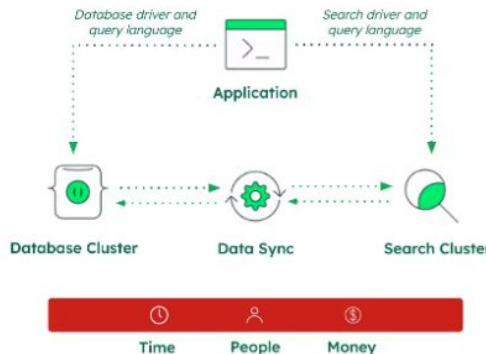
Developers need to use different query APIs for database and search and coordinate schema changes

✗ Pay the sync tax

Requires its own systems and skills. Recovering sync errors can consume 10% of a developers time

✗ Operational overhead

More to provision, secure, upgrade, patch, back up, monitor, scale, etc.



✓ Higher developer productivity

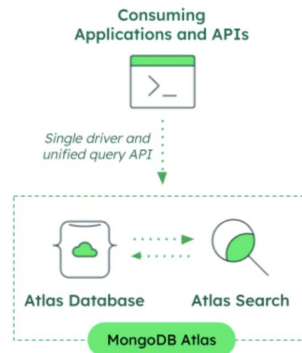
Build database and search features with a unified driver and query API

✓ Simplified data architecture

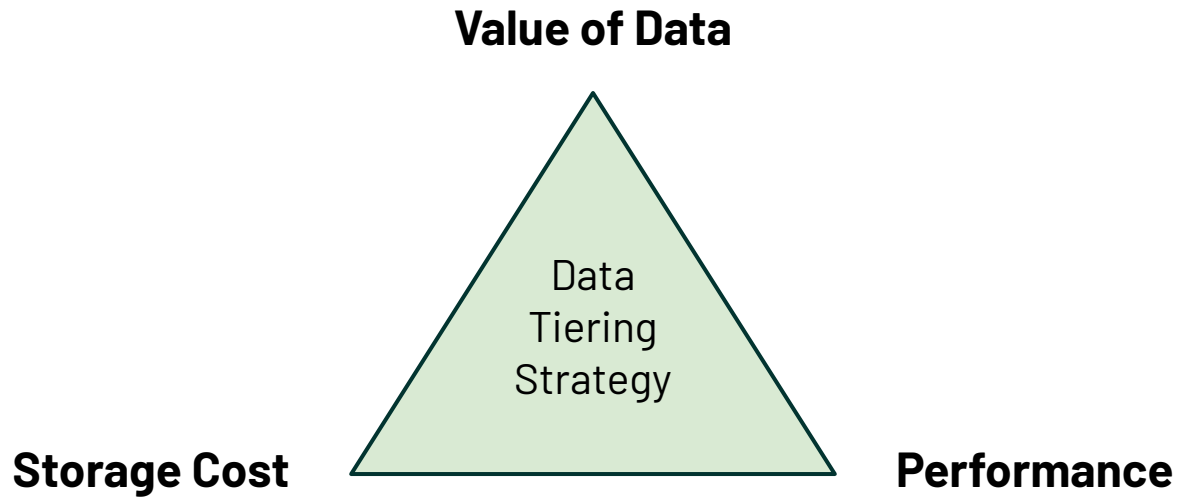
Automatic data synchronization, even as your data and schema changes

✓ Fully managed platform

Get the security, performance, and reliability of Atlas



Need to Balance Cost, Performance and Value

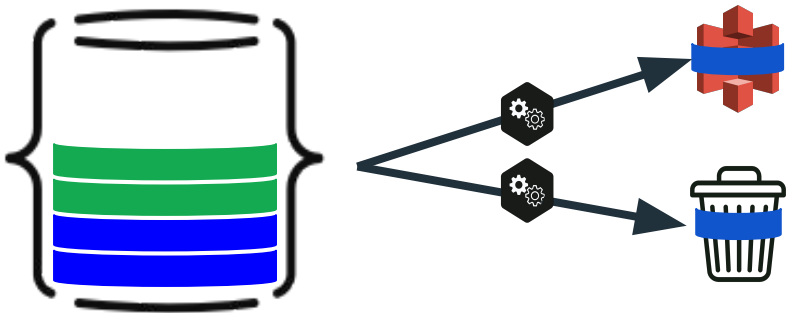


Migrate Motion: Drastically reduce TCO

Customer's Current State

Self-Managed
MongoDB Deployment

Manually exporting
or deleting data

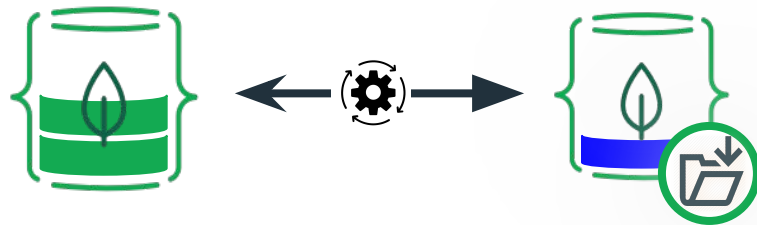


- **Significantly overpaying**
- Manual processes to export and delete data
- Difficult to query data stored in Cloud Object Storage

Future State w/ Atlas + OA

Managed Atlas
Deployment

Automated data archiving
to Managed S3 bucket



- **Drastically reduced TCO**
- Automated process to export and delete data
- Easily query data stored in Cloud Object Storage



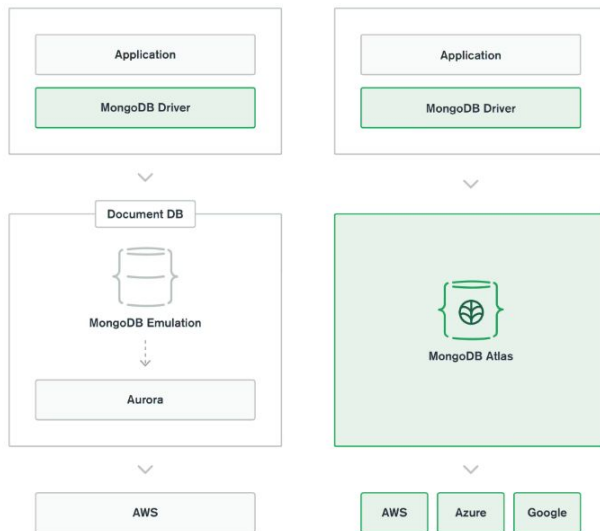
Quanto tempo em queda de produtividade por incompatibilidade do DocumentDB com MongoDB?



Amazon DocumentDB is a NoSQL JSON document database service with a limited degree of compatibility with MongoDB.

DocumentDB is not based on the MongoDB server. Rather it emulates the MongoDB API, and runs on top of Amazon's Aurora backend platform. This creates significant architectural constraints, functionality limitations and broken compatibility.

DocumentDB claims to support the MongoDB 4.0 API, which implies that it is at parity with MongoDB v4.0, released back in June 2018. However this is partially true, as the majority of MongoDB 4.0 differentiating features are not supported by DocumentDB. In addition, compatibility testing reveals it **fails over 66% of the MongoDB API correctness tests**. Applications written for MongoDB will need to be re-written to work with Amazon DocumentDB.



Interested in up-to-date results on DocumentDB's compatibility with the MongoDB API? Get the latest results at [Is DocumentDB Really MongoDB?](#)

The key differences between DocumentDB and MongoDB's on-demand, elastic, and fully managed [Atlas service](#) are summarized below.

[Link](#)